

Predicting League of Legends Match Outcomes Using Early-Game Team Statistics

Yuwen Cen

Research question. Can early-game team statistics predict whether the blue team wins a League of Legends match, and which in-game advantages are most associated with winning?

Abstract

This project studies whether early-game team statistics can be used to predict match outcomes in League of Legends. After cleaning the original dataset of 24,225 matches, I constructed difference-based features comparing blue-team and red-team advantages. Exploratory analysis showed strong separation between wins and losses for gold, experience, kills, damage, and tower pressure. Logistic regression achieved the best overall performance among the tested models, with accuracy of 0.764 and AUC of 0.845. Regularized logistic regression, Random Forest, and XGBoost produced very similar results, suggesting that the available features contain a strong but limited early-game signal. The main interpretation is that early economic and combat advantages are highly predictive, while more complex models did not substantially outperform the simpler, interpretable logistic regression model.

1. Context and Motivation

League of Legends is a team-based strategy game where match outcomes are influenced by both measurable in-game statistics and less directly observed factors such as team coordination, champion matchups, decision-making, and mechanical skill. In professional broadcasts, viewers often see live win-probability estimates that summarize the state of the game. This project is motivated by a similar idea: using early-game statistics to estimate the probability that one team will win.

The task is framed as a binary classification problem. The response variable is `blueWin`, where 1 indicates that the blue team won and 0 indicates that it lost. The predictors are early-game team statistics observed before the match outcome is known to avoid potential data leakage.

2. Data Preprocessing and Cleaning

The original dataset contained 24,225 observations and 30 columns. The initial cleaning step removed an identifier column, `matchId`, because it does not contain predictive information for future matches. Empty unnamed columns were also removed. The response variable was nearly balanced, corresponding to about 50.5% losses and 49.5% wins.

3. Feature Engineering

The most important design decision was to convert raw blue-team and red-team statistics into relative advantage features. In a competitive match, the absolute number of kills, gold, or objectives matters less than the difference between the two teams. For example, a blue team with 28,000 gold is not necessarily ahead

unless the red team has less gold. Therefore, the main feature set was constructed using difference variables such as `gold_diff`, `xp_diff`, `kill_diff`, `tower_diff`, `dragon_diff`, `herald_diff`, `plate_diff`, `damage_diff`, `ward_diff`, `minion_diff`, and `jungle_diff`.

This feature engineering approach also improves interpretability. Positive values mean the blue team is ahead, negative values mean the red team is ahead, and values near zero mean the game state is relatively even. Later in the project, I also tested ratio-based features, such as `gold_ratio` and `xp_ratio`, to see whether proportional advantage adds predictive information beyond simple differences.

4. Exploratory Data Analysis

The exploratory analysis focused on whether early-game advantages differ systematically between blue-team wins and losses. Group means showed large directional differences: when the blue team lost, `gold_diff` averaged about -2385 and `xp_diff` averaged about -1694. When the blue team won, `gold_diff` averaged about 2525 and `xp_diff` averaged about 1638. Similar patterns appeared for kills, towers, dragons, and damage.

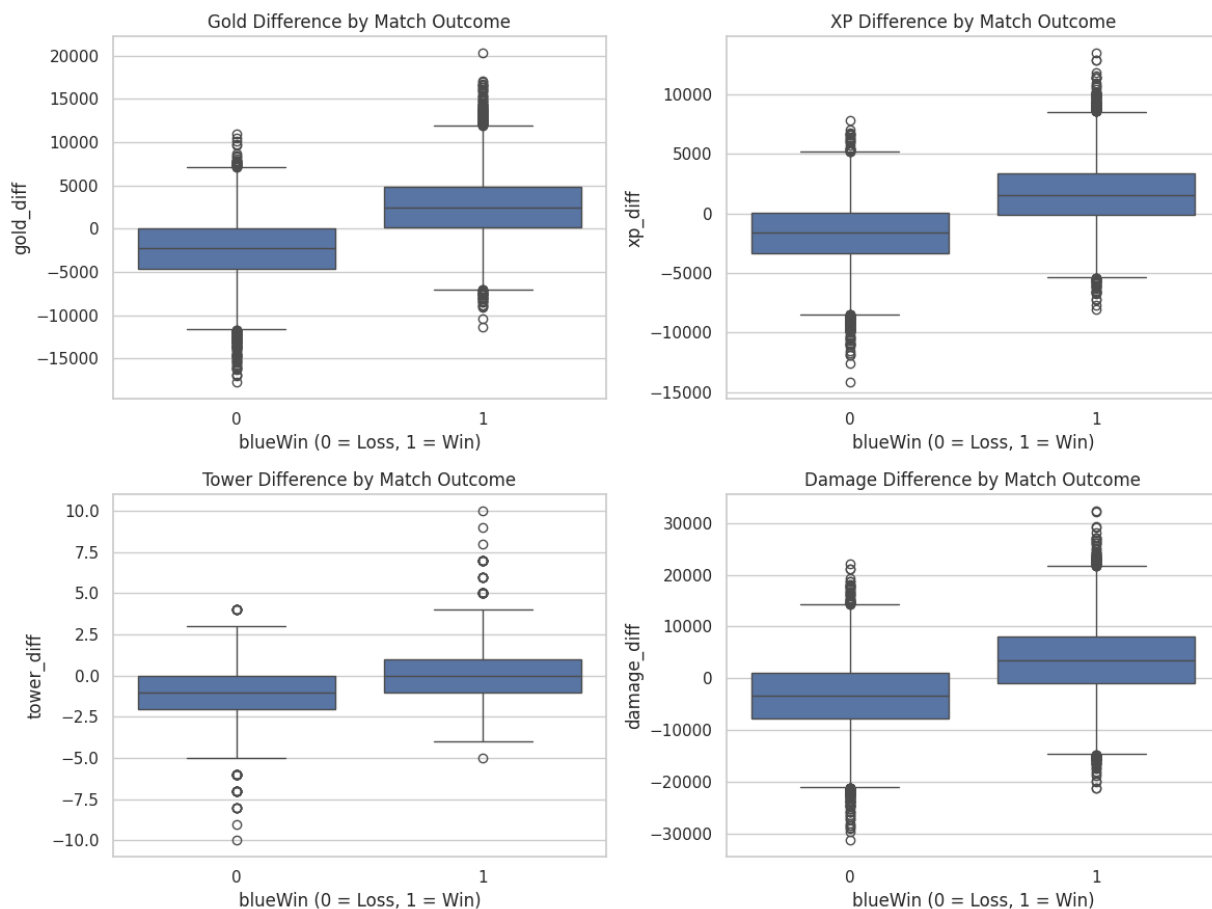


Figure 1. Distribution of key difference features by match outcome.

Figure 1 shows a clear and consistent separation between wins and losses. For gold, XP, damage, and tower differences, winning matches generally have positive median values, while losing matches have negative medians. This supports the main modeling assumption that relative early-game advantages are informative predictors of match outcome.

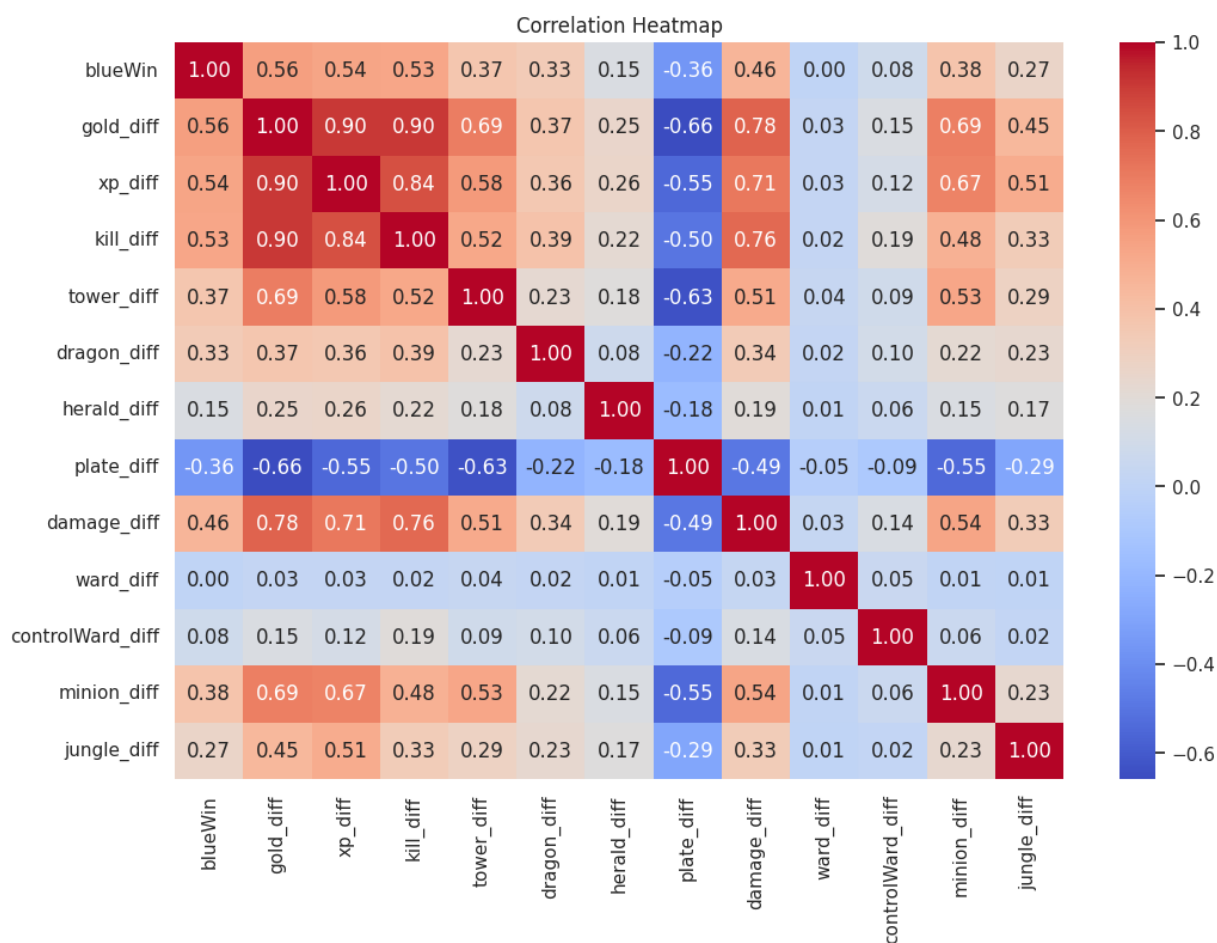


Figure 2. Correlation heatmap among outcome and engineered features.

The correlation heatmap reveals one important finding in the dataset: gold, XP, kills, and damage are strongly correlated with each other. This is expected because gaining kills and farming minions often increases both gold and experience, and stronger items or levels can then lead to more damage and more future kills. This collinearity shaped the modeling strategy. Logistic regression was used for interpretability, while Random Forest and XGBoost were used to test whether nonlinear models could extract additional signal from interactions among these correlated variables.

5. Modeling Setup and Evaluation Metrics

The dataset was split into training and test sets using an 80/20 stratified split. Stratification preserves the near-balanced class distribution in both sets. The final split contained 19380 training observations and 4845 test observations. All reported performance values in the notebook were based on the held-out test set.

The main evaluation metrics were accuracy and AUC. Accuracy measures the proportion of correctly classified matches, while AUC measures how well the model separates wins from losses across different classification thresholds. Precision, recall, and F1-score were also used to check whether the model was balanced across the two outcome classes. Since this is not a heavily imbalanced problem, no special resampling method was necessary.

6. Model Building and Evolution

6.1 Logistic Regression Baseline

The first model was logistic regression. This model is appropriate because the response is binary and the coefficients provide a direct interpretation of the direction and relative strength of each predictor. Standardization was applied before fitting, so the coefficients can be compared on a more meaningful scale.

The logistic regression model achieved accuracy of 0.7641 and AUC of 0.845. The classification report showed balanced performance across the two classes, with precision around 0.77 for wins and recall around 0.75 for wins. This indicates that the model is not simply favoring one class; instead, it is learning a meaningful relationship between early-game advantage and match result.

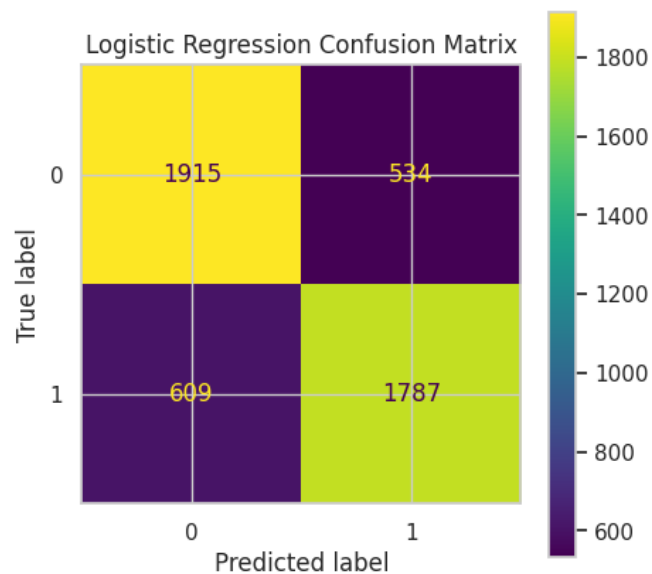


Figure 3. Logistic regression confusion matrix.

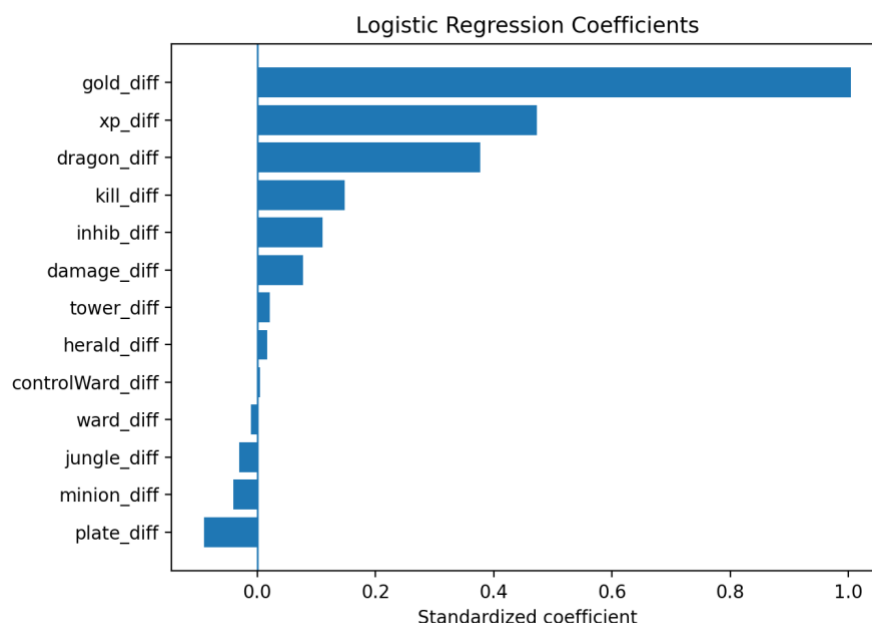


Figure 4. Logistic regression coefficient ranking.

The coefficient ranking shows that `gold_diff` is the strongest positive predictor, followed by `xp_diff` and `dragon_diff`. This means that, holding other standardized variables constant, a larger blue-side gold advantage has the largest positive association with blue-team win probability. XP and dragon control also provide important information, while some highly correlated variables contribute less once gold and XP are already included. This is a useful example of why coefficient interpretation should be done carefully in the presence of collinearity.

6.2 Regularized Logistic Regression

To test whether the logistic model could be improved through feature shrinkage or feature selection, I fitted Ridge Logistic Regression, Lasso Logistic Regression, and Elastic Net Logistic Regression using cross-validation. Ridge helps stabilize coefficients under collinearity, Lasso can shrink unimportant coefficients to zero, and Elastic Net combines both approaches.

The regularized models performed almost identically to the baseline logistic regression. Ridge achieved accuracy of 0.7641 and AUC of 0.8450; Lasso achieved accuracy of 0.7626 and AUC of 0.8447; Elastic Net achieved accuracy of 0.7626 and AUC of 0.8445. This suggests that regularization was useful as a robustness check, but it did not reveal a substantially more accurate model. The similar performance also suggests that the baseline logistic regression was not severely overfitting.

6.3 Random Forest

The next model was Random Forest, which was included to capture nonlinear relationships and interactions that logistic regression may miss. Random Forest builds many decision trees on bootstrap samples and averages their predictions. At each split, it randomly considers a subset of predictors, which reduces correlation among trees and improves generalization.

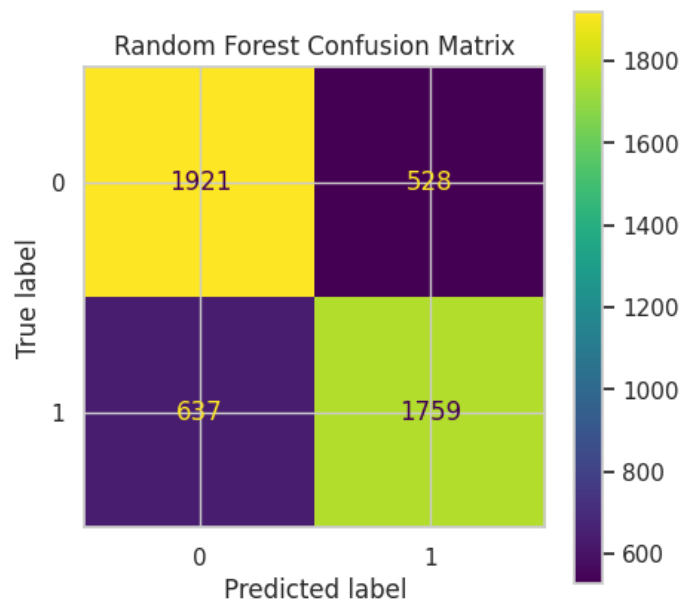


Figure 5. Random Forest confusion matrix.

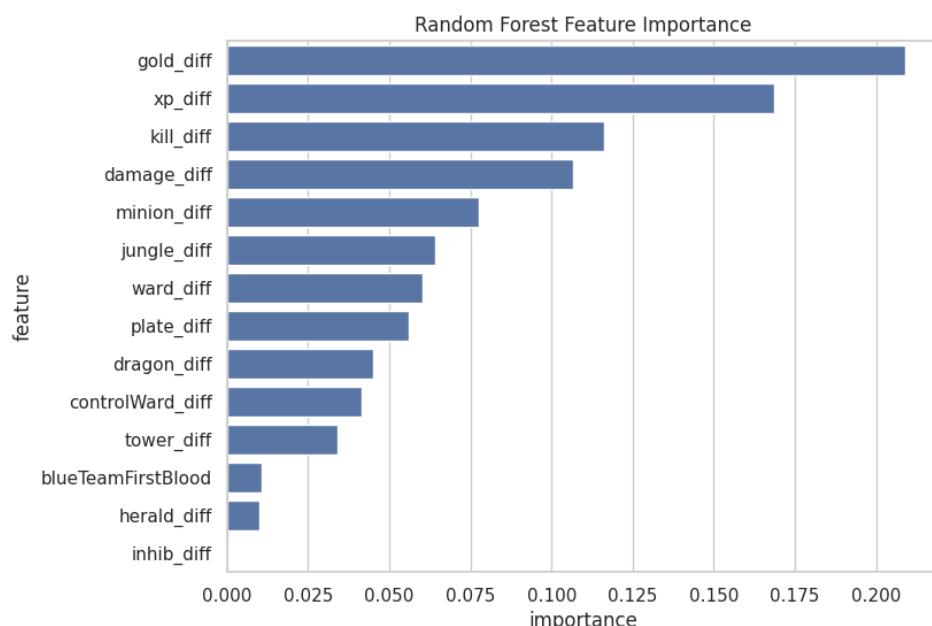


Figure 6. Random Forest feature importance.

The Random Forest model achieved accuracy of 0.7595 and AUC of 0.8384, slightly below the logistic regression baseline. Its feature importance ranking again placed gold_diff, xp_diff, kill_diff, and damage_diff near the top. This reinforces the main substantive conclusion: early-game economic and combat advantage dominates the prediction task. However, the fact that Random Forest did not outperform logistic regression suggests that nonlinear interactions were not strong enough.

6.4 XGBoost and Feature Extension

XGBoost was tested as a more flexible tree-based ensemble model. It sequentially builds trees that correct the errors of previous trees, often producing strong performance on tabular prediction problems. The model achieved accuracy of about 0.7600 and AUC of 0.8428, which was close to but still slightly below logistic regression. To further solve multicollinearity, I then experimented with ratio-based features, also yielding similar performance.

7. Model Comparison and Results

Model	Accuracy	AUC
Ridge Logistic (L2)	0.7641	0.8450
Logistic Regression	0.7641	0.8450
Lasso Logistic (L1)	0.7626	0.8447
Elastic Net Logistic	0.7626	0.8445
XGBoost	0.7600	0.8428
Tuned XGBoost	0.7579	0.8390
Random Forest	0.7595	0.8384

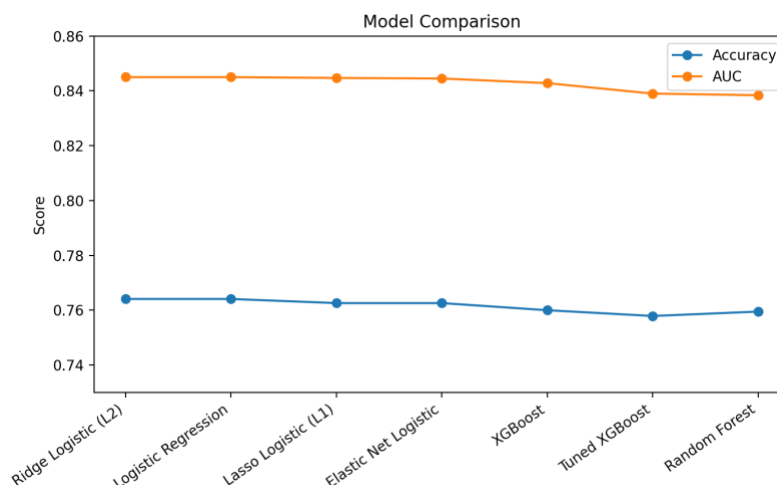


Figure 7. Accuracy and AUC comparison across models.

The model comparison shows that logistic-regression-based models performed best overall. Standard logistic regression and Ridge Logistic Regression tied for the highest accuracy and AUC. Lasso and Elastic Net were only slightly lower. Random Forest and XGBoost, despite being more flexible, did not outperform the simpler linear model, suggesting the linearity nature of the data.

8. Conclusion

The finding is that early-game advantages in gold, experience, kills, damage, and objectives are strongly associated with winning. Gold difference is the most influential predictor across multiple models, which makes practical sense because gold directly converts into items and combat strength. XP difference also matters because levels unlock stronger abilities and base statistics. Dragon and tower-related variables capture objective control and map pressure, which are strategically meaningful beyond raw combat metrics.

Logistic regression confirmed that the relationship was strong enough to achieve about 76% test accuracy. Regularization did not improve results, suggesting that the main signal was already stable. Random Forest and XGBoost did not provide meaningful gains, which implies that the predictive ceiling may be driven more by missing information than by model choice. Important unobserved factors include champion skills interaction, weapon choices, team coordination, strategy, etc.

As a result, the early-game statistics can be useful as a live reference tool, especially for audiences watching professional games. However, it should be used cautiously for coaching or training decisions. The analysis is associational rather than causal. A feature being predictive does not prove that directly increasing that feature will cause a team to win, because many variables are jointly determined by broader game quality and decision-making.

Appendix

Rusovan, K. (2024). *League of Legends SoloQ matches at 15 minutes 2024* [Data set]. Kaggle. Retrieved May 3, 2026, from <https://www.kaggle.com/datasets/karlorusovan/league-of-legends-soloq-matches-at-10-minutes-2024>